



المدرسة الوطنية للعلوم التطبيقية
École Nationale des Sciences Appliquées d'Oujda
+212 054 24 00 00 | 054 24 00 00 | 054 24 00 00

Université Mohammed Premier
École Nationale des Sciences Appliquées d'Oujda
Filière Génie Informatique – GI3

Rapport de Mini-Projet

[OSMAR]

Réalisé par :
Marwa MARHRANE
Oussama HAJJI

Encadrante : Mme Douae EL HILA

Module : Développement Java IHM
Année : 2025/2026

Remerciements

Nous tenons tout d'abord à exprimer nos sincères remerciements à **Mme ELFARISSI Ilhame** pour la qualité de son enseignement du module *Développement Java IHM*. Sa pédagogie, sa clarté et son engagement nous ont permis d'acquérir des bases solides en programmation orientée objet et en conception d'interfaces graphiques, des compétences qui ont été déterminantes pour la réalisation de ce mini-projet.

Nous adressons également nos sincères remerciements et notre profonde gratitude à **Mme EL HILA Douae**, notre encadrante pour ce mini-projet, pour la qualité de son enseignement, sa disponibilité constante et la pertinence de ses orientations tout au long de la réalisation de ce travail. Sa rigueur, son sens de la pédagogie et son accompagnement attentif nous ont permis d'approfondir nos connaissances en programmation orientée objet et en développement d'interfaces graphiques avec JavaFX, tout en nous offrant les clés nécessaires pour mener ce projet à bien avec autonomie et confiance. Nous la remercions également pour sa patience face à nos interrogations, ainsi que pour l'intérêt sincère qu'elle a porté à l'avancement de notre travail, faisant de cette expérience une occasion à la fois enrichissante et formatrice, tant sur le plan technique qu'humain.

Que ce modeste travail soit le témoignage de notre reconnaissance et de notre profond respect envers elles deux.

Table des matières

Remerciements	1
Introduction	5
1 Présentation du projet	6
1.1 Description de l'application	6
1.2 Besoins fonctionnels	7
1.3 Besoins non fonctionnels	7
2 Conception	8
2.1 Architecture MVC	8
2.2 Diagramme de classes UML	8
2.3 Diagramme de cas d'utilisation	10
3 Environnement de travail	11
3.1 Outils et technologies utilisés	11
3.2 Structure du projet	11
4 Répartition des tâches	13
4.1 Tâches de l'étudiant 1 – [Oussama Hajji]	13
4.2 Tâches de l'étudiant 2 – [Marwa Marhrane]	13
5 Explication du code	15
5.1 Connexion à la base de données – DB_connection.java	15
5.2 Les classes modèles	15
5.3 Les classes DAO	16
5.4 Les contrôleurs	16
5.5 Les fichiers FXML	17
5.6 Fonctionnalité spécifique au projet	17
6 Interfaces de l'application	19
6.1 Écran d'accueil et authentification	19
6.2 Page d'accueil	19
6.3 Catalogue des ingrédients	19
6.4 Composition et passage de commande	21
6.5 Historique des commandes	21
Conclusion	23
Références	24

Table des figures

2.1	Architecture MVC de l'application OSMAR	9
2.2	Diagramme de classes UML de l'application OSMAR	9
2.3	Diagramme de cas d'utilisation de l'application OSMAR	10
6.1	Écran de bienvenue	19
6.2	Écran de connexion	19
6.3	Page d'accueil après connexion	20
6.4	Catalogue – vue Toutes	20
6.5	Catalogue – Matières naturelles	20
6.6	Catalogue – Matières synthétiques	20
6.7	Catalogue – Fixateurs	20
6.8	Catalogue – Solvants/Supports	21
6.9	Détail d'un ingrédient (Pivoine Royale)	21
6.10	Récapitulatif de la commande	21
6.11	Choix de la quantité du flacon	21
6.12	Résumé de la composition (étape 4)	22
6.13	Choix du mode de paiement	22
6.14	Saisie de l'adresse de livraison	22
6.15	Confirmation de commande	22
6.16	Historique des commandes	22

Liste des tableaux

4.1	Tâches réalisées par l'étudiant 1	13
4.2	Tâches réalisées par l'étudiant 2	14

Introduction

Avec l'essor du numérique, les applications de gestion occupent aujourd'hui une place centrale dans la plupart des secteurs d'activité, en permettant d'automatiser, de simplifier et de personnaliser des processus autrefois réalisés de manière manuelle. Cette évolution s'accompagne d'une exigence croissante en matière d'ergonomie et d'expérience utilisateur, ce qui place le développement d'interfaces graphiques au cœur des préoccupations des développeurs.

C'est dans cette optique que s'inscrit le module *Développement Java IHM*, qui vise à initier les étudiants à la conception et à la réalisation d'applications graphiques robustes à l'aide des technologies Java. Dans le cadre de ce module, il nous a été demandé de réaliser, en binôme, un mini-projet mettant en pratique l'ensemble des notions abordées durant le semestre, depuis la modélisation des besoins jusqu'à l'implémentation d'une interface fonctionnelle, en passant par la gestion d'une base de données.

C'est ainsi que nous avons conçu et développé l'application **OSMAR**, dont la présentation détaillée fera l'objet de la section suivante de ce rapport.

Ce rapport retrace l'ensemble de la démarche suivie pour la réalisation de ce projet. Il débute par une présentation de l'application et de ses besoins, avant d'aborder les aspects de conception à travers l'architecture adoptée et les différents diagrammes UML. Il décrit ensuite l'environnement de travail ainsi que la répartition des tâches entre les membres du groupe, suivi d'une explication détaillée du code source. Enfin, le rapport présente les différentes interfaces de l'application avant de conclure sur le bilan de ce travail et les perspectives d'amélioration envisagées.

Chapitre 1

Présentation du projet

1.1 Description de l'application

C'est dans ce contexte que s'inscrit notre projet, intitulé OSMAR, une application de gestion et de création de parfums sur mesure. Contrairement aux parfums conventionnels proposés en magasin, OSMAR offre à l'utilisateur la possibilité de sélectionner librement les ingrédients qu'il souhaite intégrer dans la composition de son propre parfum, lui permettant ainsi de devenir acteur du processus créatif. L'application a été conçue pour s'adresser à tout type d'utilisateur, sans distinction de genre, et propose des compositions adaptées aussi bien aux parfums féminins, masculins.

L'objectif principal d'OSMAR est de simplifier considérablement le processus de production de parfums personnalisés. Traditionnellement, un client souhaitant composer son propre parfum devait se déplacer physiquement jusqu'à l'entreprise, énumérer la liste des ingrédients désirés, puis attendre la confirmation de sa commande – une démarche aussi coûteuse en temps qu'en effort. OSMAR supprime cette contrainte en mettant à la disposition du client une application desktop intuitive, développée avec JavaFX, à travers laquelle il peut accéder, depuis son ordinateur, à l'ensemble des services nécessaires à la création et à la commande de son parfum, sans aucun déplacement.

Le parcours utilisateur débute par une étape d'authentification : si le client possède déjà un compte, il se connecte directement ; dans le cas contraire, il procède à une inscription rapide. Une fois authentifié, il est redirigé vers la page d'accueil, qui présente une brève description du fonctionnement général de l'application ainsi que deux options principales. La première, « Historique des commandes », permet au client de consulter, sous forme de tableau, l'ensemble des commandes qu'il a déjà passées. La seconde, « Effectuer une commande », l'amène vers la section centrale de l'application : la composition du parfum.

Dans cette section, les ingrédients proposés par l'entreprise sont organisés en quatre catégories distinctes – fixateur, matière synthétique, matière première naturelle, et solvant/support – afin de faciliter la navigation et le choix du client. Chaque ingrédient est représenté sous la forme d'une carte affichant son nom et son type ; un clic sur la carte permet d'en afficher les détails complets, notamment son prix au gramme. Le client sélectionne ainsi librement les ingrédients qu'il souhaite intégrer à sa composition.

Une fois les ingrédients choisis, le client accède, via le bouton « Ma commande », à un récapitulatif complet de sa composition, où il peut à tout moment retirer un ingrédient ou en ajouter de nouveaux avant de valider son choix. La confirmation de la commande le conduit ensuite vers une étape de personnalisation de la quantité de parfum souhaitée, exprimée par la taille du flacon, comprise entre un minimum de 30 ml et un maximum de 100 ml. Le client renseigne enfin l'adresse de livraison souhaitée puis confirme définitivement sa commande. Un message de confirmation s'affiche alors pour signaler que la commande a bien été enregistrée.

et sauvegardée. Enfin, lorsque la commande est prête, un email est automatiquement envoyé au client afin de l'en informer.

1.2 Besoins fonctionnels

Les besoins fonctionnels identifiés pour l'application OSMAR, qui découlent directement du scénario d'utilisation décrit précédemment, sont les suivants :

- Permettre au client de créer un compte (inscription) ou de se connecter à un compte existant (authentification).
- Afficher une page d'accueil présentant brièvement le principe de fonctionnement de l'application.
- Permettre au client de consulter l'historique de ses commandes précédentes sous forme de tableau.
- Afficher la liste des ingrédients disponibles, classés par catégorie (fixateur, matière synthétique, matière première naturelle, solvant/support).
- Présenter chaque ingrédient sous forme de carte (nom, type) et afficher ses détails complets (description, prix au gramme) au clic.
- Permettre au client de sélectionner, ajouter ou retirer des ingrédients de sa composition.
- Générer un récapitulatif de la commande en cours regroupant tous les ingrédients sélectionnés.
- Permettre au client de choisir la quantité de parfum souhaitée (entre 30 ml et 100 ml, via la taille du flacon).
- Permettre au client de renseigner l'adresse de livraison de sa commande.
- Confirmer et enregistrer la commande dans la base de données.
- Envoyer un email de notification au client une fois la commande prête.

1.3 Besoins non fonctionnels

Au-delà des fonctionnalités proposées, l'application OSMAR doit également répondre à un certain nombre d'exigences de qualité garantissant une expérience utilisateur satisfaisante et un fonctionnement fiable :

- **Ergonomie et simplicité** : l'interface doit être intuitive et accessible à tout utilisateur, sans nécessiter de compétences techniques particulières.
- **Performance et réactivité** : la navigation entre les différentes sections (ingrédients, commande, historique) doit être fluide et rapide.
- **Fiabilité** : les commandes saisies doivent être correctement enregistrées et sauvegardées sans perte de données.
- **Sécurité** : les données personnelles et les informations de connexion des clients doivent être protégées.
- **Disponibilité** : l'application doit être accessible à tout moment pour permettre au client de passer commande.
- **Portabilité** : développée avec JavaFX, l'application doit pouvoir s'exécuter sur différents systèmes d'exploitation sans modification majeure.
- **Maintenabilité** : le code doit être structuré selon une architecture claire (MVC) afin de faciliter les évolutions futures de l'application.

Chapitre 2

Conception

2.1 Architecture MVC

L'application OSMAR repose sur une architecture MVC (Modèle-Vue-Contrôleur), un patron de conception largement répandu dans le développement d'interfaces graphiques en Java, qui permet de séparer clairement la logique métier, la présentation et la gestion des interactions utilisateur. Cette séparation facilite la maintenance, les tests et l'évolution indépendante de chaque couche.

La couche **Modèle** correspond au package `model`, qui regroupe les classes représentant les entités métier de l'application : `Client`, `Commande`, ainsi que la hiérarchie des ingrédients (`Ingredient`, classe abstraite, et ses quatre sous-classes `Fixateur`, `Matiere_synthetique`, `Matiere_Premiere_naturelle` et `Solvant_Support`). Cette couche ne contient aucune logique d'affichage ; elle se limite à représenter les données et les règles de calcul propres au domaine, telles que la méthode `calculer_prix()` qui détermine le prix d'un ingrédient à partir de sa quantité et de son type.

La couche **Vue** est constituée des fichiers FXML situés dans `resources/view/fxml`, un par écran de l'application (`LoginView`, `CatalogView`, `CartView`, `QuantityView`, etc.), accompagnés d'une feuille de style commune `theme.css` qui assure la cohérence visuelle de l'ensemble de l'application.

La couche **Contrôleur** regroupe, dans le package `controller`, une classe contrôleur par écran (`LoginController`, `CatalogController`, `CartController`, `QuantityController`, etc.), chargée de réagir aux actions de l'utilisateur, de mettre à jour la vue et de faire appel aux classes du modèle et aux DAO. La navigation entre les différents écrans est centralisée dans la classe utilitaire `SceneManager`, tandis que la classe `Session` conserve les données devant persister d'un écran à l'autre (client connecté, commande en cours de construction, ingrédient sélectionné).

Enfin, la persistance des données est assurée par un ensemble de classes **DAO** (Data Access Object), regroupées dans le package `dao`, chacune responsable des opérations CRUD sur une table de la base de données MySQL `osmar_db` (`ClientDAO`, `CommandeDAO`, `FixateurDAO`, etc.), toutes s'appuyant sur une connexion centralisée fournie par `DB_connection`. Cette organisation en couches, illustrée par la figure 2.1, garantit une bonne séparation des responsabilités tout au long de l'application.

2.2 Diagramme de classes UML

La figure 2.2 présente le diagramme de classes UML de la couche modèle de l'application OSMAR. Il met en évidence la hiérarchie d'héritage des ingrédients : la classe abstraite

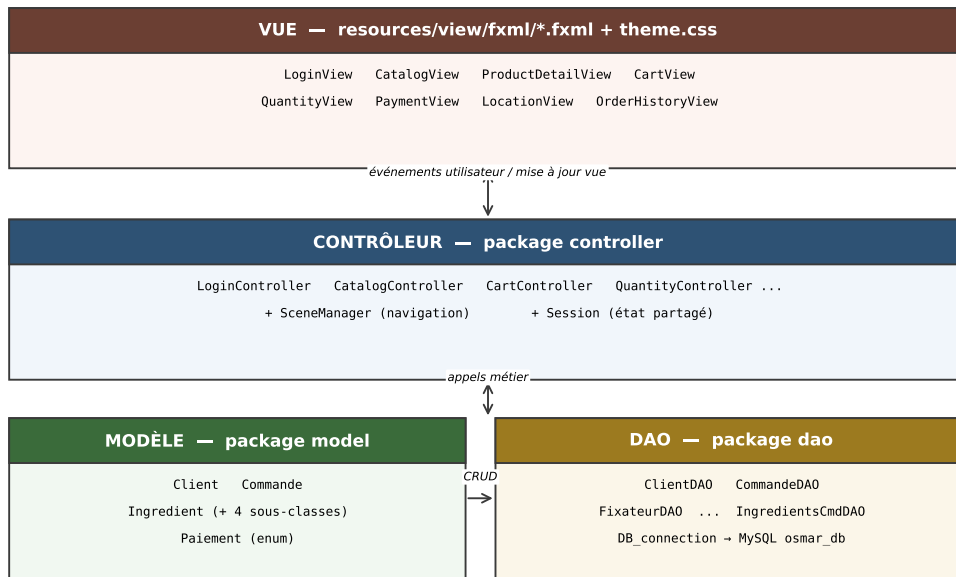


Figure 2.1: Architecture MVC de l'application OSMAR

Ingredient définit les attributs et le comportement communs à tous les ingrédients (nom, quantité, disponibilité, prix, ainsi que la méthode abstraite `calculer_prix()`), tandis que les quatre sous-classes **Fixateur**, **Matiere_synthetique**, **Matiere_Premiere_naturelle** et **Solvant_Support** en héritent et redéfinissent cette méthode selon le type d'ingrédient, chaque type étant associé à une énumération qui fixe son prix au gramme.

La classe **Commande** agrège un ensemble d'ingrédients (`elements_cmd`, de type `Set<Ingredient>`), représentant la composition du parfum choisie par le client, ainsi que la quantité de flacon souhaitée et les dates de commande et de livraison. La classe **Client** est associée à plusieurs commandes et possède un mode de paiement préféré, représenté par l'énumération **Paiement**.

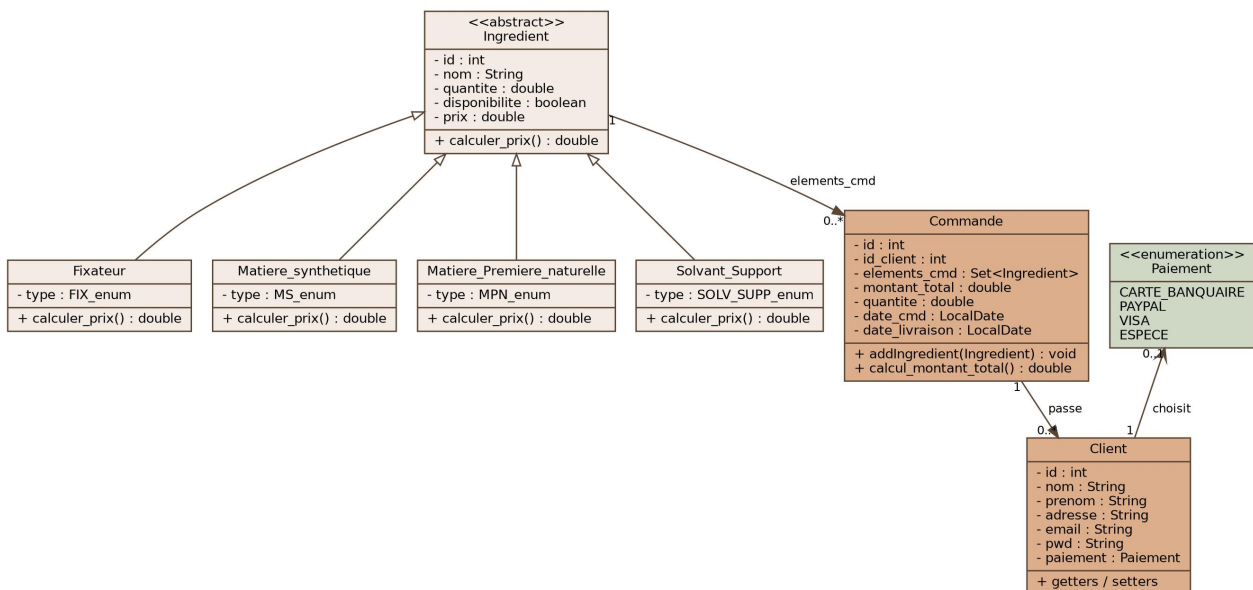


Figure 2.2: Diagramme de classes UML de l'application OSMAR

2.3 Diagramme de cas d'utilisation

La figure 2.3 présente le diagramme de cas d'utilisation de l'application OSMAR. Le seul acteur identifié est le **Client**, qui interagit avec le système pour créer un compte, se connecter, consulter l'historique de ses commandes, ou composer un nouveau parfum. Ce dernier cas d'utilisation se décompose, via des relations «*include*», en plusieurs étapes successives correspondant au parcours réel de l'application : parcourir le catalogue d'ingrédients classés par catégorie, consulter le détail d'un ingrédient, gérer le panier de sa composition, choisir la quantité du parfum, sélectionner un mode de paiement, renseigner l'adresse de livraison, puis confirmer définitivement la commande.

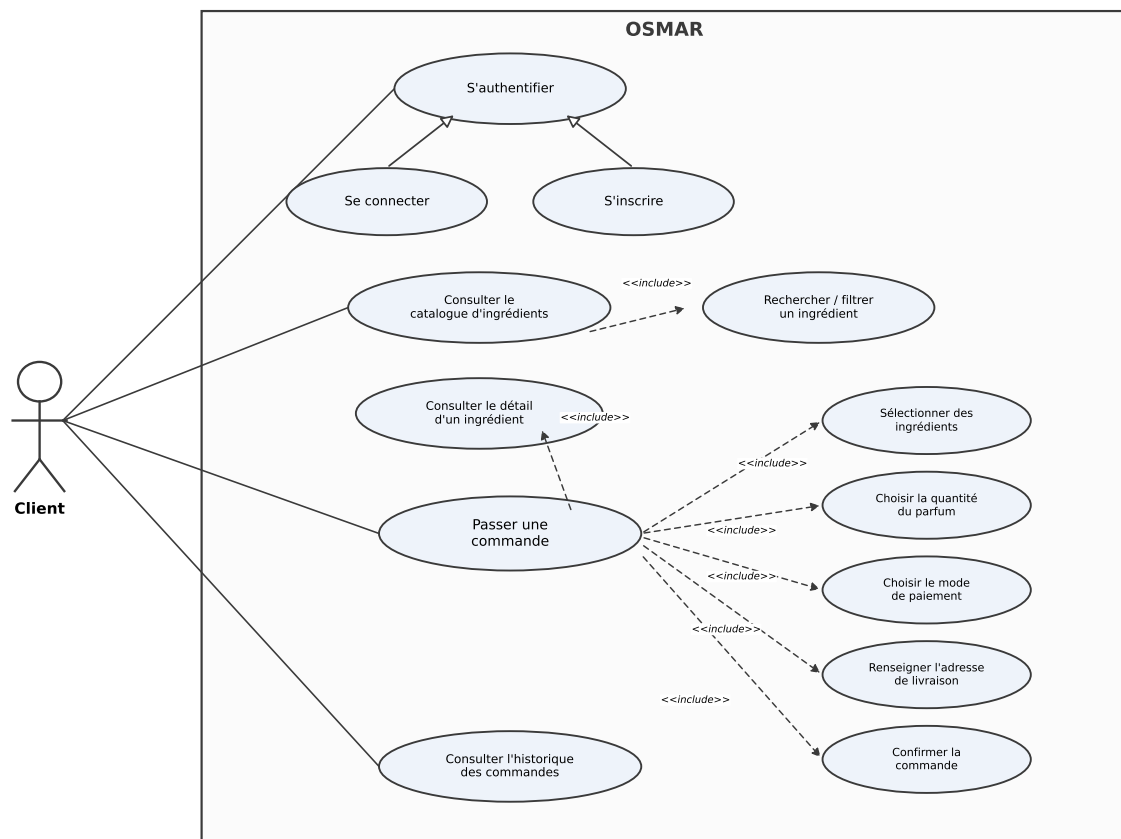


Figure 2.3: Diagramme de cas d'utilisation de l'application OSMAR

Chapitre 3

Environnement de travail

3.1 Outils et technologies utilisés

Le développement de l'application OSMAR s'appuie sur un ensemble d'outils et de technologies propres à l'écosystème Java moderne. Le langage utilisé est **Java 21**, associé au framework **JavaFX** (version 23.0.2, modules `javafx-controls` et `javafx-fxml`) pour la conception de l'interface graphique, les vues étant décrites de manière déclarative à travers des fichiers FXML accompagnés d'une feuille de style CSS commune. La gestion du projet, des dépendances et de la compilation est assurée par **Maven**, à l'aide du plugin `javafx-maven-plugin` qui permet de lancer l'application directement via la commande `mvn javafx:run`. La persistance des données repose sur un système de gestion de base de données relationnelle **MySQL**, auquel l'application se connecte par l'intermédiaire du pilote JDBC `mysql-connector-j`. Le développement a été réalisé sous l'environnement de développement intégré **IntelliJ IDEA**, et la fiabilité du code a été renforcée par des tests unitaires écrits avec le framework **JUnit 5**.

3.2 Structure du projet

Le projet suit l'arborescence standard d'un projet Maven, organisée autour du dossier `src/main`, lui-même divisé en deux parties : `java`, qui contient le code source réparti par package selon l'architecture MVC présentée au chapitre précédent, et `resources`, qui regroupe les vues FXML ainsi que la feuille de style CSS. La structure complète du projet est la suivante :

```
osmar/
|-- pom.xml
'-- src/
    '-- main/
        |-- java/
        |   |-- com/example/osmar/           (point d'entree et utilitaires)
        |   |   |-- MainApp.java
        |   |   |-- SceneManager.java
        |   |   |-- Session.java
        |   |   '-- SignUpDraft.java
        |   |
        |   |-- controller/                  (un controleur par ecran)
        |   |   |-- WelcomeController.java
        |   |   |-- LoginController.java
        |   |   |-- SignUpStepOneController.java
        |   |   '-- SignUpStepTwoController.java
```

```

| | |-- HomeController.java
| | |-- CatalogController.java
| | |-- ProductDetailController.java
| | |-- CartController.java
| | |-- QuantityController.java
| | |-- CompositionSummaryController.java
| | |-- PaymentMethodController.java
| | |-- LocationController.java
| | |-- OrderConfirmationController.java
| | |-- OrderHistoryController.java
| | |-- IngredientCardFactory.java
| | '-- CommandeRow.java
|
| |-- dao/                                (acces aux donnees - JDBC)
| | |-- DB_connection.java
| | |-- ClientDAO.java
| | |-- CommandeDAO.java
| | |-- IngredientsCmdDAO.java
| | |-- IngredientTypeResolver.java
| | |-- FixateurDAO.java
| | |-- MatiereSynthetiqueDAO.java
| | |-- MatierePremiereNaturelleDAO.java
| | '-- SolvantSupportDAO.java
|
| '-- model/                              (entites metier)
|   |-- Client.java
|   |-- Commande.java
|   |-- Paiement.java
|   |-- Ingredient.java
|   |-- Fixateur.java                    + FIX_enum.java
|   |-- Matiere_synthetique.java        + MS_enum.java
|   |-- Matiere_Premiere_naturelle.java + MPN_enum.java
|   '-- Solvant_Support.java            + SOLV_SUPP_enum.java
|
|-- resources/view/
|   |-- fxml/                            (14 ecrans)
|   | |-- WelcomeView.fxml               |-- QuantityView.fxml
|   | |-- LoginView.fxml                 |-- CompositionSummaryView.fxml
|   | |-- SignUpStepOneView.fxml         |-- PaymentMethodView.fxml
|   | |-- SignUpStepTwoView.fxml         |-- LocationView.fxml
|   | |-- HomeView.fxml                  |-- OrderConfirmationView.fxml
|   | |-- CatalogView.fxml               '-- OrderHistoryView.fxml
|   | |-- ProductDetailView.fxml
|   |-- CartView.fxml
|   '-- css/
|       '-- theme.css

```

Chapitre 4

Répartition des tâches

4.1 Tâches de l'étudiant 1 – [Oussama Hajji]

Tâche réalisée	Fichier(s) concerné(s)	Statut
Conception et implémentation de la couche Modèle (entités métier, hiérarchie des ingrédients, énumérations de prix)	Client.java, Commande.java, Paiement.java, Ingredient.java, Fixateur.java, FIX_enum.java, Matiere_synthetique.java, MS_enum.java, Matiere_Premiere_naturelle.java, MPN_enum.java, Solvant_Support.java, SOLV_SUPP_enum.java	Terminé
Développement de la couche Contrôleur (logique des écrans, navigation, gestion du panier et du parcours de commande)	WelcomeController.java, LoginController.java, SignUpStepOneController.java, SignUpStepTwoController.java, HomeController.java, CatalogController.java, ProductDetailController.java, CartController.java, QuantityController.java, CompositionSummaryController.java, PaymentMethodController.java, LocationController.java, OrderConfirmationController.java, OrderHistoryController.java, IngredientCardFactory.java, CommandeRow.java	Terminé
Développement de la couche DAO (accès aux données et persistance JDBC vers la base osmar_db)	DB_connection.java, ClientDAO.java, CommandeDAO.java, IngredientsCmdDAO.java, IngredientTypeResolver.java, FixateurDAO.java, MatiereSynthetiqueDAO.java, MatierePremiereNaturelleDAO.java, SolvantSupportDAO.java	Terminé

Table 4.1: Tâches réalisées par l'étudiant 1

4.2 Tâches de l'étudiant 2 – [Marwa Marhrane]

Tâche réalisée	Fichier(s) concerné(s)	Statut
Rédaction du rapport du mini-projet	rapport.tex	Terminé
Conception de la couche Vue (écrans FXML et feuille de style associée)	WelcomeView.fxml, LoginView.fxml, SignUpStepOneView.fxml, SignUpStepTwoView.fxml, HomeView.fxml, CatalogView.fxml, ProductDetailView.fxml, CartView.fxml, QuantityView.fxml, CompositionSummaryView.fxml, PaymentMethodView.fxml, LocationView.fxml, OrderConfirmationView.fxml, OrderHistoryView.fxml, theme.css	Terminé

Table 4.2: Tâches réalisées par l'étudiant 2

Chapitre 5

Explication du code

5.1 Connexion à la base de données – DB_connection.java

L'accès à la base de données MySQL `osmar_db` est centralisé dans une unique classe utilitaire, `DB_connection`, plutôt que d'être dupliqué dans chaque DAO. Sa classe est déclarée `final` avec un constructeur privé, et expose une unique méthode statique `getConnection()` qui ouvre une connexion JDBC à partir de l'URL, de l'utilisateur et du mot de passe de la base. Toute classe DAO appelle cette méthode plutôt que d'ouvrir sa propre connexion, ce qui garantit que les paramètres de connexion ne vivent qu'à un seul endroit du code.

```
1 public final class DB_connection {
2
3     private static final String URL = "jdbc:mysql://localhost:3306/
        osmar_db";
4     private static final String USER = "root";
5     private static final String PASSWORD = "root";
6
7     private DB_connection() { }
8
9     public static Connection getConnection() throws SQLException {
10         return DriverManager.getConnection(URL, USER, PASSWORD);
11     }
12 }
```

5.2 Les classes modèles

Le package `model` regroupe les entités métier de l'application. La classe abstraite `Ingredient` factorise les attributs communs à tout ingrédient (`nom`, `quantite`, `disponibilite`, `prix`) ainsi qu'une méthode abstraite `calculer_prix()`, redéfinie différemment par chacune des quatre sous-classes `Fixateur`, `Matiere_synthetique`, `Matiere_Premiere_naturelle` et `Solvant_Support`. Chaque sous-classe associe l'ingrédient à une énumération (`FIX_enum`, `MS_enum`, `MPN_enum`, `SOLV_SUPP_enum`) qui fixe le prix au gramme propre à chaque type, illustrant un usage classique du polymorphisme :

```
1 public class Fixateur extends Ingredient {
2
3     private FIX_enum type;
4
5     @Override
6     public double calculer_prix() {
7         return prix = this.quantite * type.getPrixParGramme();
8     }
9 }
```

```
9 }
```

La classe `Commande` représente la composition de parfum en cours de création : elle agrège un ensemble d'ingrédients (`elements_cmd`, de type `Set<Ingredient>`), la quantité de flacon choisie ainsi que les dates de commande et de livraison. La classe `Client` porte les informations du compte utilisateur, et l'énumération `Paielement` liste les modes de paiement disponibles.

5.3 Les classes DAO

Chaque classe du package `dao` encapsule les opérations CRUD (création, lecture, mise à jour, suppression) relatives à une table de la base de données, en s'appuyant systématiquement sur des `PreparedStatement` afin de prévenir toute injection SQL, et sur le bloc `try-with-resources` pour garantir la fermeture automatique des connexions. Le cas le plus représentatif est `CommandeDAO`, qui ne se limite pas à insérer la commande elle-même : la composition (`elements_cmd`) étant stockée dans une table de jointure séparée (`ingredients_cmd`), la méthode `insertWithIngredients()` orchestre l'insertion de la commande puis, pour chaque ingrédient sélectionné, délègue à `IngredientsCmdDAO` l'enregistrement de la ligne correspondante – le type exact de l'ingrédient étant déterminé dynamiquement par la classe utilitaire `IngredientTypeResolver`.

```
1 public int insertWithIngredients(Commande commande) {
2     int idCmd = insert(commande);
3     if (idCmd == -1) {
4         return -1;
5     }
6     for (Ingredient ingredient : commande.getElementsCmd()) {
7         String typeIngredient = IngredientTypeResolver.resolve(
8             ingredient);
9         ingredientsCmdDAO.insert(idCmd, ingredient.getID(),
10             ingredient.getQuantite(), typeIngredient);
11     }
12     return idCmd;
13 }
```

5.4 Les contrôleurs

Chaque écran de l'application possède son propre contrôleur dans le package `controller`, implémentant l'interface `Initializable` et reliant les composants graphiques annotés `@FXML` aux actions correspondantes (recherche, filtrage, navigation, validation). Le passage d'un écran à l'autre est centralisé dans `SceneManager.switchTo(...)`, tandis que la classe `Session` conserve, sous forme de champs statiques, les données devant survivre à ces changements d'écran : le client connecté (`currentClient`), la commande en cours de construction (`currentCommande`) et l'ingrédient sélectionné dans le catalogue (`selectedIngredient`).

```
1 public static Commande startNewCommande() {
2     if (currentClient == null) {
3         throw new IllegalStateException(
4             "Impossible de demarrer une commande sans client connecte.");
5     }
6     LocalDate dateLivraison = LocalDate.now().plusDays(7);
7     currentCommande = new Commande(currentClient.getID(),
```

```

8         dateLivraison.getYear(), dateLivraison.getMonthValue(),
9         dateLivraison.getDayOfMonth());
10     return currentCommande;
11 }

```

5.5 Les fichiers FXML

Les vues sont décrites de manière déclarative dans des fichiers FXML, situés dans `resources/view/fxml` et reliés à leur contrôleur via l'attribut `fx:controller`. Chaque composant destiné à être manipulé en Java est identifié par un attribut `fx:id`, et l'ensemble des écrans partage la même feuille de style `theme.css`. La vue `CatalogView.fxml`, par exemple, structure l'écran de composition du parfum à l'aide d'un `AnchorPane` racine contenant un en-tête (titre, champ de recherche, bouton panier), une barre de filtres par catégorie, puis un `ScrollPane` dont le contenu est généré dynamiquement par le contrôleur :

```

1 <AnchorPane fx:id="rootPane" fx:controller="controller.CatalogController
  " ...>
2   <VBox ...>
3     <HBox alignment="CENTER_LEFT" spacing="20.0">
4       <TextField fx:id="searchField" promptText="Rechercher une
        essence..." />
5       <Button fx:id="cartButton" text="Ma Commande" />
6     </HBox>
7     <HBox spacing="12.0">
8       <Button fx:id="catAllButton" text="Toutes" styleClass="
        category-chip-active" />
9       <Button fx:id="catFixateurButton" text="Fixateurs"
        styleClass="category-chip" />
10      <!-- ... autres categories ... -->
11    </HBox>
12    <ScrollPane fitToWidth="true">
13      <VBox fx:id="productGridContainer" />
14    </ScrollPane>
15  </VBox>
16 </AnchorPane>

```

5.6 Fonctionnalité spécifique au projet

La fonctionnalité la plus caractéristique d'OSMAR est le calcul dynamique et personnalisé du prix d'un parfum, combiné à un catalogue interactif filtrable en temps réel. Lorsqu'un client choisit un ingrédient, son prix est recalculé instantanément en multipliant la quantité saisie (en grammes) par le prix au gramme propre à son type exact, grâce au polymorphisme de la méthode `calculer_prix()` présentée en section 5.2. Du côté du catalogue, le contrôleur `CatalogController` recompose entièrement la grille de produits à chaque frappe dans le champ de recherche ou changement de catégorie active, en combinant un filtre sur le type d'ingrédient et un filtre textuel sur le nom :

```

1 List<Ingredient> filtered = allIngredients.stream()
2   .filter(this::matchesFamilyFilter)
3   .filter(ing -> query.isEmpty()
4     || ing.getNom().toLowerCase().contains(query))
5   .toList();

```

Cette approche, certes simple, permet une expérience fluide et réactive pour le client : il compose librement son parfum en associant filtrage, recherche, calcul de prix au gramme et gestion d'un panier persistant à travers les écrans, jusqu'à la confirmation finale de la commande.

Chapitre 6

Interfaces de l'application

6.1 Écran d'accueil et authentification

L'application s'ouvre sur un écran de bienvenue (figure 6.1) présentant OSMAR et invitant le client à commencer son parcours. Il est ensuite redirigé vers la fenêtre de connexion (figure 6.2), où il saisit son adresse e-mail et son mot de passe, avec un lien vers la création de compte pour les nouveaux utilisateurs.

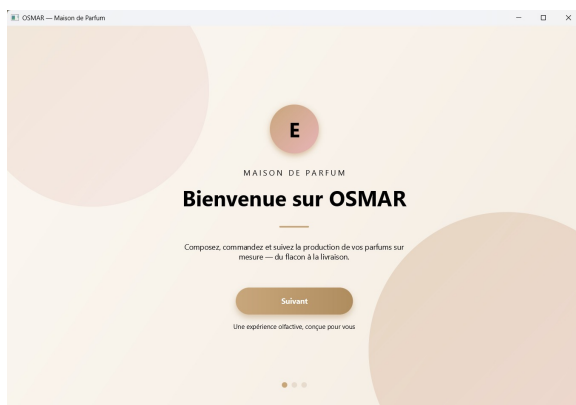


Figure 6.1: Écran de bienvenue

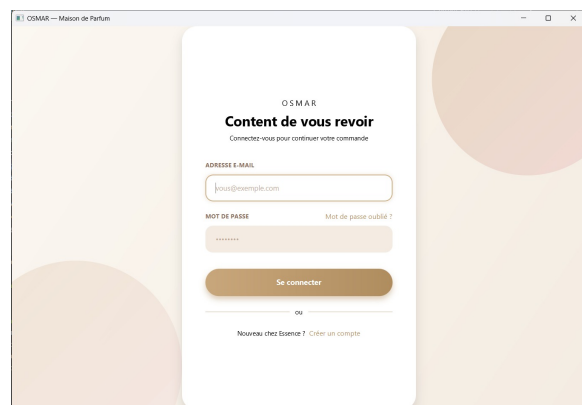


Figure 6.2: Écran de connexion

6.2 Page d'accueil

Une fois connecté, le client arrive sur la page d'accueil (figure 6.3) qui l'accueille par son prénom et lui présente le concept de l'application en trois étapes claires. Deux boutons lui permettent soit de démarrer une nouvelle commande, soit de consulter ses commandes passées.

6.3 Catalogue des ingrédients

Le catalogue (figures 6.4 à 6.9) est l'interface centrale de l'application. Il affiche l'ensemble des ingrédients sous forme de cartes, filtrables par catégorie via une barre de filtres (Toutes, Matières naturelles, Matières synthétiques, Fixateurs, Solvants/Supports) et recherchables par nom. Un clic sur une carte ouvre une fenêtre de détail affichant la description complète de l'ingrédient, sa disponibilité et son prix au gramme, avec un bouton « Ajouter à la commande ».

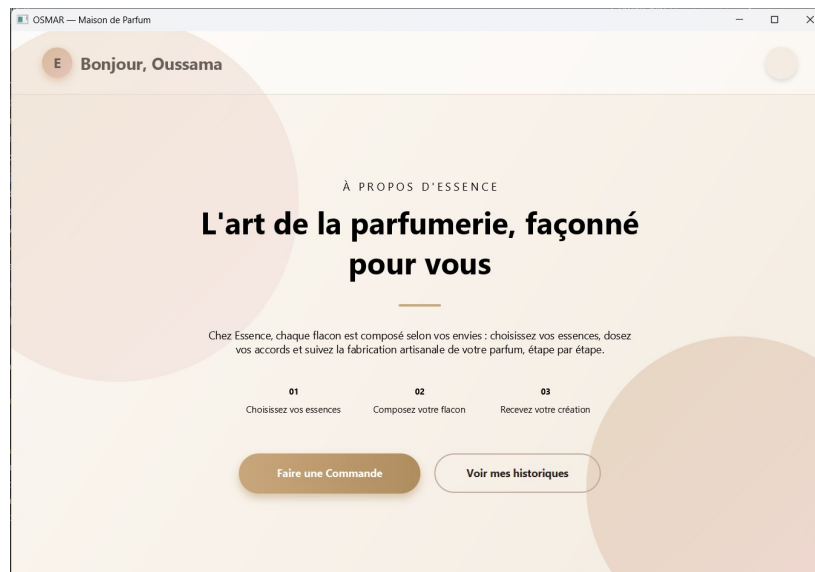


Figure 6.3: Page d'accueil après connexion

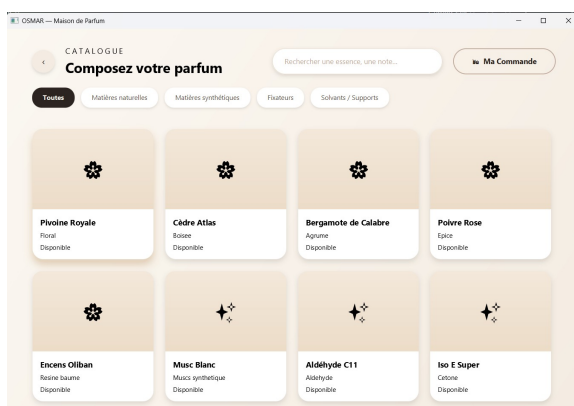


Figure 6.4: Catalogue – vue Toutes

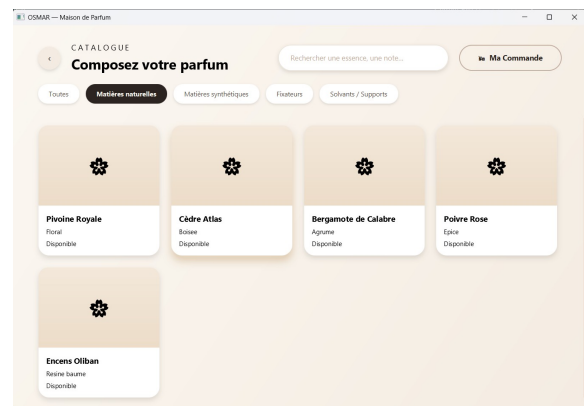


Figure 6.5: Catalogue – Matières naturelles

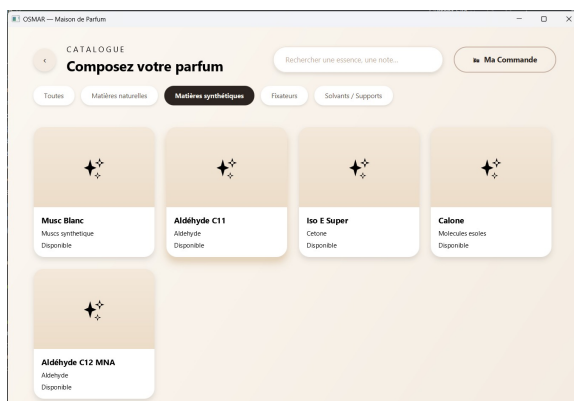


Figure 6.6: Catalogue – Matières synthétiques

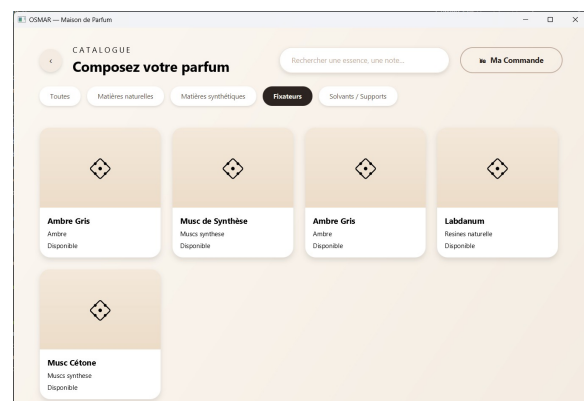


Figure 6.7: Catalogue – Fixateurs

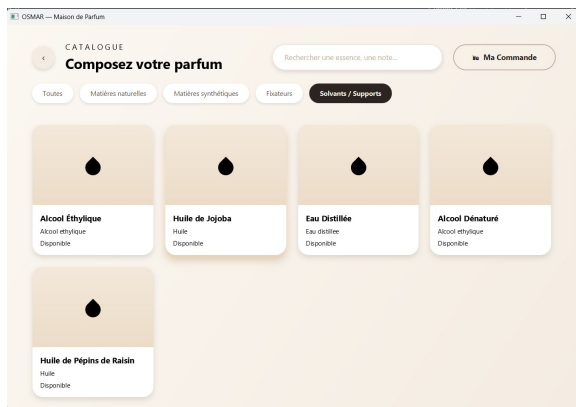


Figure 6.8: Catalogue – Solvants/Supports

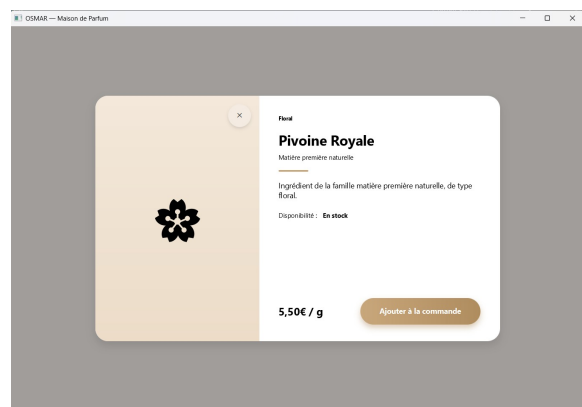


Figure 6.9: Détail d'un ingrédient (Pivoine Royale)

6.4 Composition et passage de commande

Le processus de commande se déroule en plusieurs étapes guidées. L'écran « Ma Commande » (figure 6.10) récapitule les ingrédients sélectionnés avec leur prix unitaire et le sous-total. Le client choisit ensuite la quantité de son flacon via un slider (figure 6.11), puis valide sa composition (figure 6.12), choisit son mode de paiement (figure 6.13) et renseigne son adresse de livraison (figure 6.14).

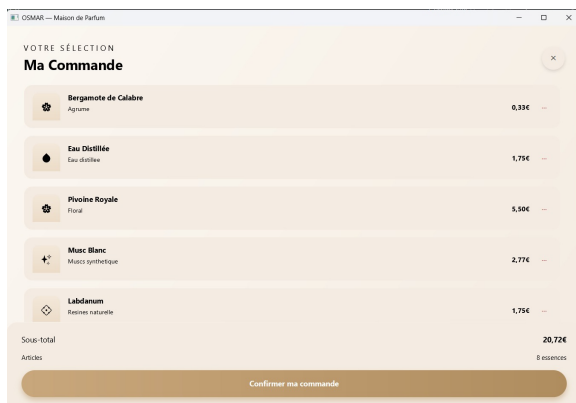


Figure 6.10: Récapitulatif de la commande

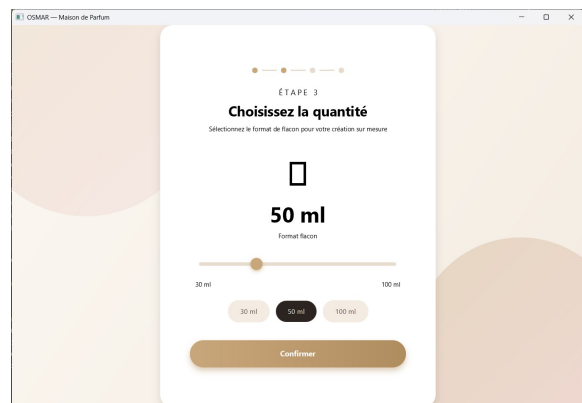


Figure 6.11: Choix de la quantité du flacon

6.5 Historique des commandes

L'écran d'historique (figure 6.16) présente, sous forme de tableau, l'ensemble des commandes passées par le client, avec pour chacune le numéro de commande, la date, la composition, la quantité, le montant total et le statut.

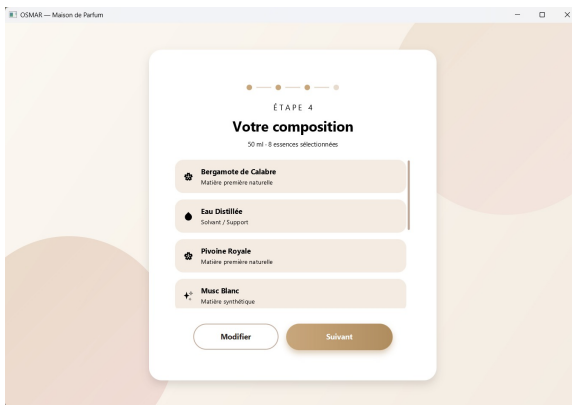


Figure 6.12: Résumé de la composition (étape 4)

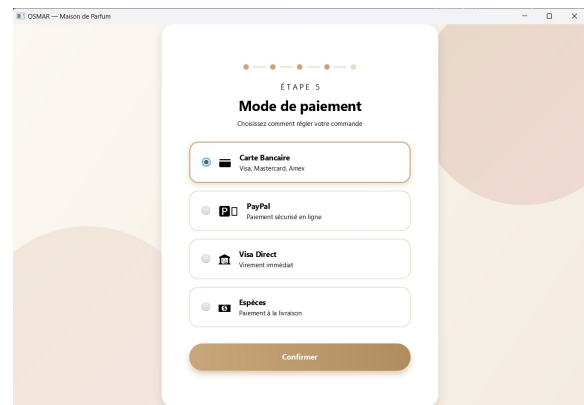


Figure 6.13: Choix du mode de paiement

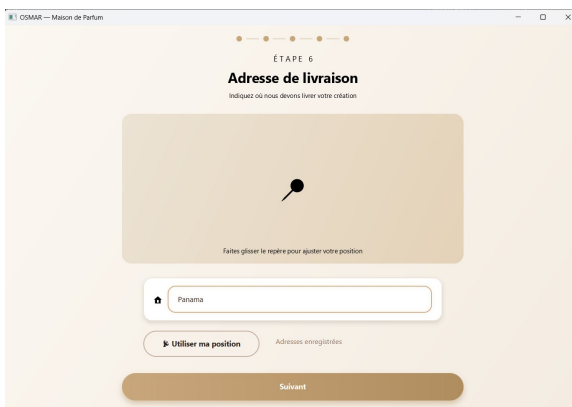


Figure 6.14: Saisie de l'adresse de livraison

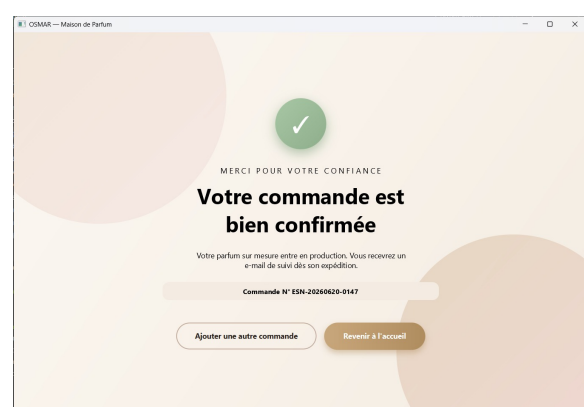


Figure 6.15: Confirmation de commande

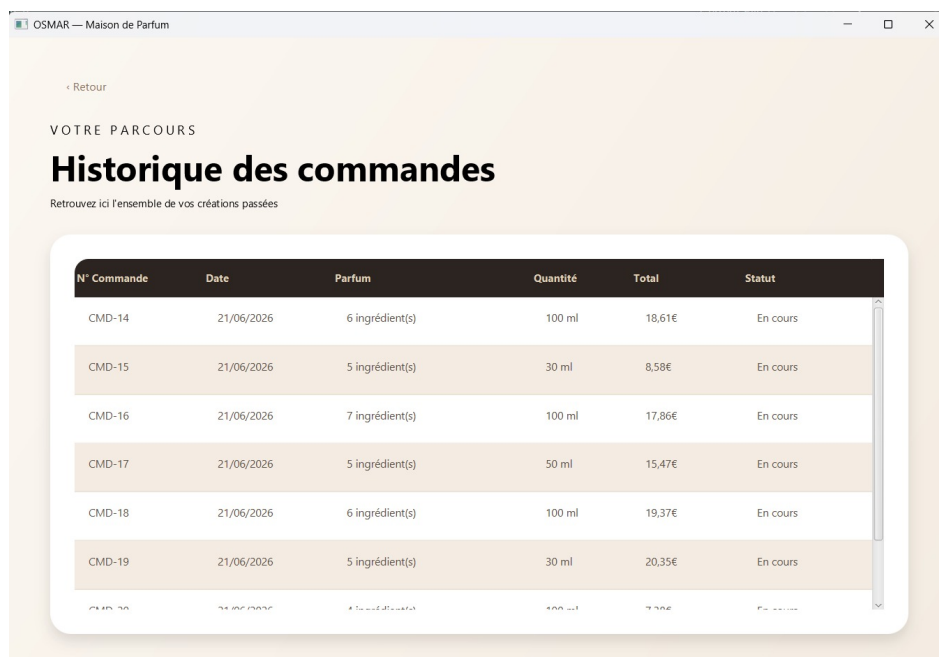


Figure 6.16: Historique des commandes

Conclusion

Ce mini-projet nous a permis de concevoir et de développer OSMAR, une application JavaFX complète permettant à un client de composer un parfum entièrement personnalisé, depuis la sélection des ingrédients jusqu'à la confirmation de sa commande. L'architecture MVC adoptée, appuyée sur une base de données MySQL et une couche d'accès aux données structurée, a permis d'obtenir une application cohérente, où chaque couche – modèle, vue, contrôleur, persistance – assume un rôle clairement délimité.

Au-delà du résultat technique, ce projet a été l'occasion de mettre en pratique l'ensemble des notions abordées dans le module *Développement Java IHM* : programmation orientée objet, conception d'interfaces graphiques avec JavaFX et FXML, gestion d'une base de données relationnelle, et organisation du travail en binôme autour d'un dépôt de code partagé.

Plusieurs pistes restent envisageables pour enrichir l'application : la mise en place effective de l'envoi d'email de confirmation, l'ajout d'un espace d'administration pour gérer le catalogue d'ingrédients et les stocks, l'intégration d'un véritable module de paiement en ligne, ou encore le portage de l'application vers une version web ou mobile. Ce travail constitue ainsi une base solide, à la fois fonctionnelle et évolutive, pour de futurs développements.

Références

- Documentation officielle JavaFX 23 : <https://openjfx.io/javadoc/23/>
- Guide des composants JavaFX (Oracle) : https://docs.oracle.com/javafx/2/ui_controls/jfxpub-ui_controls.htm
- Documentation du langage Java 21 (Oracle) : <https://docs.oracle.com/en/java/javase/21/>
- Référence JDBC – connexion à MySQL : <https://dev.mysql.com/doc/connector-j/en/>
- Documentation MySQL 8.0 : <https://dev.mysql.com/doc/refman/8.0/en/>
- Maven – guide d'utilisation officiel : <https://maven.apache.org/guides/>
- Plugin javafx-maven-plugin : <https://github.com/openjfx/javafx-maven-plugin>
- JUnit 5 – guide utilisateur : <https://junit.org/junit5/docs/current/user-guide/>
- IntelliJ IDEA – documentation : <https://www.jetbrains.com/help/idea/>
- Code source du projet OSMAR : <https://github.com/Oussama-exe/Gestion-De-Production-De>